

TCP simple:

Server:

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/socket.h>
#include <netinet/in.h>
#define PORT 9000

int main() {
    char message[256] = "pineapple on pizza? hard pass, bro!";
    char buffer[256];
    // Create a socket
    int server_socket;
    server_socket = socket(AF_INET, SOCK_STREAM, 0);
    // Setup socket address
    struct sockaddr_in server_address;
    server_address.sin_family = AF_INET;
    server_address.sin_port = htons(PORT);
    server_address.sin_addr.s_addr = INADDR_ANY;
    // Bind the socket to our specified IP and port
    bind(server_socket, (struct sockaddr *)&server_address,
sizeof(server_address));
    // Listen for connections
    listen(server_socket, 5);
    // Accept a connection from a client
    int client_socket;
    client_socket = accept(server_socket, NULL, NULL);
    recv(client_socket, buffer, sizeof(buffer), 0);
    printf("Message from client: %s\n", buffer);
    send(client_socket, message, sizeof(message), 0);
    printf("Message sent from server.\n");
    close(client_socket);
    close(server_socket);
    return 0;
}
```

Client:

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/socket.h>
#include <netinet/in.h>
#define PORT 9000

int main() {
    char request[256] = "Yo server, pineapple on pizza?";
    char buffer[256];
    // Create a socket
    int sock = socket(AF_INET, SOCK_STREAM, 0);
    // Setup socket address
    struct sockaddr_in server_address;
    server_address.sin_addr.s_addr = INADDR_ANY;
    server_address.sin_family = AF_INET;
    server_address.sin_port = htons(PORT);
    // Connect to the server
    if (connect(sock, (struct sockaddr *)&server_address,
sizeof(server_address)) < 0) {
        perror("Connection failed");
        exit(1);
    }
    printf("Connected to server.\n");
    send(sock, request, sizeof(request), 0);
    printf("Message sent from client.\n");
    int valread = recv(sock, buffer, sizeof(buffer), 0);
    printf("Message from server: %s\n", buffer);
    close(sock);
    return 0;
}
```

UDP (simple):

Server:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/socket.h>
#include <netinet/in.h>
#define PORT 9000

int main() {
    char buffer[256];
    char message[256] = "Hello from UDP server!";
    int sockfd;
    struct sockaddr_in server_addr, client_addr;
    socklen_t addr_size = sizeof(client_addr);
    // Create socket
    sockfd = socket(AF_INET, SOCK_DGRAM, 0);
    // Setup server address
    server_addr.sin_family = AF_INET;
    server_addr.sin_port = htons(PORT);
    server_addr.sin_addr.s_addr = INADDR_ANY;
    // Bind socket
    bind(sockfd, (struct sockaddr*)&server_addr, sizeof(server_addr));
    // Receive message from client
    int n = recvfrom(sockfd, buffer, sizeof(buffer)-1, 0, (struct
sockaddr*)&client_addr, &addr_size);
    buffer[n] = '\0'; // Null-terminate
    printf("Message from client: %s\n", buffer);
    // Send reply
    sendto(sockfd, message, strlen(message)+1, 0, (struct
sockaddr*)&client_addr, addr_size);
    close(sockfd);
    return 0;
}
```

Client:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/socket.h>
#include <netinet/in.h>
#define PORT 9000

int main() {
    char buffer[256];
    char request[256] = "Hello UDP server, how are you?";
    int sockfd;
    // Create socket
    sockfd = socket(AF_INET, SOCK_DGRAM, 0);
    struct sockaddr_in server_addr;
    socklen_t addr_size = sizeof(server_addr);
    // Server address setup
    server_addr.sin_family = AF_INET;
    server_addr.sin_port = htons(PORT);
    server_addr.sin_addr.s_addr = INADDR_ANY;
    // Send message
    sendto(sockfd, request, strlen(request)+1, 0, (struct
sockaddr*)&server_addr, addr_size);
    // Receive reply
    recvfrom(sockfd, buffer, sizeof(buffer), 0, (struct
sockaddr*)&server_addr, &addr_size);
    printf("Message from server: %s\n", buffer);
    close(sockfd);
    return 0;
}
```

Multi clients:

```
fork();
```

Server:

```
#include <arpa/inet.h>
#include <netinet/in.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <sys/socket.h>
#include <sys/types.h>    // For pid_t type
#include <unistd.h>       // For fork(), close()

#define PORT 4444
#define BUFFER_SIZE 1024

int main() {
    int sockfd, ret;
    struct sockaddr_in serverAddr;
    int clientSocket;
    struct sockaddr_in cliAddr;
    socklen_t addr_size;
    pid_t childpid;
    int opt = 1;
    char buffer[BUFFER_SIZE];

    // Creates a TCP socket id from IPV4 family
    sockfd = socket(AF_INET, SOCK_STREAM, 0);
    if (sockfd < 0) {
        printf("Error in connection.\n");
        exit(1);
    }

    printf("Server Socket is created.\n");

    // Allow address reuse
    if (setsockopt(sockfd, SOL_SOCKET, SO_REUSEADDR, &opt, sizeof(opt)) <
0) {
        perror("setsockopt");
```

```

        exit(EXIT_FAILURE);
    }

    // Initializing address structure with NULL
    memset(&serverAddr, '\0', sizeof(serverAddr));
    serverAddr.sin_family = AF_INET;
    serverAddr.sin_port = htons(PORT);
    serverAddr.sin_addr.s_addr = inet_addr("127.0.0.1");

    // Binding the socket id with the socket structure
    ret = bind(sockfd, (struct sockaddr*)&serverAddr, sizeof(serverAddr));
    if (ret < 0) {
        printf("Error in binding.\n");
        exit(1);
    }

    if (listen(sockfd, 10) == 0) {
        printf("Listening...\n\n");
    }

    addr_size = sizeof(cliAddr);

    while (1) {
        // Accept client connection
        clientSocket = accept(sockfd, (struct sockaddr*)&cliAddr,
&addr_size);
        if (clientSocket < 0) {
            exit(1);
        }

        // Displaying information of connected client
        printf("Connection accepted from %s:%d\n",
inet_ntoa(cliAddr.sin_addr), ntohs(cliAddr.sin_port));

        // Create a child process for the connected client
        if ((childpid = fork()) == 0) {
            // Close the listening socket in the child process
            close(sockfd);

            while (1) {

```

```

        // Clear the buffer
        memset(buffer, 0, BUFFER_SIZE);

        // Receive message from the client
        int recv_bytes = recv(clientSocket, buffer, BUFFER_SIZE,
0);

        if (recv_bytes <= 0) {
            printf("Client disconnected.\n");
            break;
        }

        // Print the client's message
        printf("Client: %s\n", buffer);

        // Respond back to the client
        send(clientSocket, buffer, strlen(buffer), 0);
    }

    // Close the client socket and exit child process
    close(clientSocket);
    exit(0);
}

// Close the client socket in the parent process
close(clientSocket);
}

// Close the listening socket
close(sockfd);
return 0;
}

```

Client:

```

#include <arpa/inet.h>
#include <netinet/in.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <sys/socket.h>

```

```
#include <sys/types.h>
#include <unistd.h>
#define PORT 4444
#define BUFFER_SIZE 1024

int main() {
    int clientSocket, ret;
    struct sockaddr_in serverAddr;
    char buffer[BUFFER_SIZE];
    // Creating socket id
    clientSocket = socket(AF_INET, SOCK_STREAM, 0);
    if (clientSocket < 0) {
        printf("Error in connection.\n");
        exit(1);
    }
    printf("Client Socket is created.\n");

    // Initializing server address structure
    memset(&serverAddr, '\0', sizeof(serverAddr));
    serverAddr.sin_family = AF_INET;
    serverAddr.sin_port = htons(PORT);
    serverAddr.sin_addr.s_addr = inet_addr("127.0.0.1");

    // Connect to the server
    ret = connect(clientSocket, (struct sockaddr*)&serverAddr,
sizeof(serverAddr));
    if (ret < 0) {
        printf("Error in connection.\n");
        exit(1);
    }
    printf("Connected to Server.\n");

    while (1) {
        // Clear the buffer
        memset(buffer, 0, BUFFER_SIZE);

        // Input message to send to the server
        printf("Enter message: ");
        fgets(buffer, BUFFER_SIZE, stdin);
```

```

    // Send message to the server
    send(clientSocket, buffer, strlen(buffer), 0);

    // Clear the buffer to receive the server's response
    memset(buffer, 0, BUFFER_SIZE);

    // Receive the response from the server
    if (recv(clientSocket, buffer, BUFFER_SIZE, 0) < 0) {
        printf("Error in receiving data.\n");
    } else {
        printf("Server: %s\n", buffer);
    }
}

// Close the client socket
close(clientSocket);
return 0;
}

```

Thread:

Server:

```

#include <arpa/inet.h>
#include <netinet/in.h>
#include <pthread.h>    // for threads
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <sys/socket.h>
#include <unistd.h>

#define PORT 7000
#define BUFFER_SIZE 1024

// Function prototype for the thread function
void *handle_client(void *arg);

int main() {
    int server_fd, client_fd;

```

```

struct sockaddr_in server_addr, client_addr;
socklen_t addr_size;
pthread_t tid;

// Create socket
server_fd = socket(AF_INET, SOCK_STREAM, 0);
if (server_fd < 0) {
    perror("Socket creation failed");
    exit(EXIT_FAILURE);
}
printf("Server socket created.\n");

// Configure server address
server_addr.sin_family = AF_INET;
server_addr.sin_port = htons(PORT);
server_addr.sin_addr.s_addr = inet_addr("127.0.0.1");
memset(server_addr.sin_zero, '\0', sizeof(server_addr.sin_zero));

// Bind
if (bind(server_fd, (struct sockaddr *)&server_addr,
sizeof(server_addr)) < 0) {
    perror("Bind failed");
    exit(EXIT_FAILURE);
}
printf("Bind to port %d successful.\n", PORT);

// Listen
if (listen(server_fd, 10) == 0) {
    printf("Listening...\n");
} else {
    perror("Listen failed");
    exit(EXIT_FAILURE);
}

// Accept connections in a loop
while (1) {
    addr_size = sizeof(client_addr);
    client_fd = accept(server_fd, (struct sockaddr *)&client_addr,
&addr_size);
    if (client_fd < 0) {

```

```

        perror("Accept failed");
        continue;
    }

    printf("Connection accepted from %s:%d\n",
inet_ntoa(client_addr.sin_addr), ntohs(client_addr.sin_port));

    // Create a thread for each client
    int *pclient = malloc(sizeof(int));
    *pclient = client_fd;
    pthread_create(&tid, NULL, handle_client, pclient);

    // Detach thread so its resources are freed automatically
    pthread_detach(tid);
}

close(server_fd);
return 0;
}

// Thread function to handle each client
void *handle_client(void *arg) {
    int client_fd = *((int *)arg);
    free(arg);

    char buffer[BUFFER_SIZE];
    int bytes_received;

    while (1) {
        memset(buffer, 0, BUFFER_SIZE);
        bytes_received = recv(client_fd, buffer, BUFFER_SIZE, 0);

        if (bytes_received <= 0) {
            printf("Client disconnected.\n");
            close(client_fd);
            break;
        }

        printf("Client says: %s\n", buffer);
        send(client_fd, buffer, strlen(buffer), 0); // Echo back
    }
}

```

```
}

return NULL;
}
```

Client:

```
#include <arpa/inet.h>
#include <netinet/in.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <sys/socket.h>
#include <unistd.h>

#define PORT 7000
#define BUFFER_SIZE 1024

int main() {
    int client_fd;
    struct sockaddr_in server_addr;
    char buffer[BUFFER_SIZE];

    // Create socket
    client_fd = socket(AF_INET, SOCK_STREAM, 0);
    if (client_fd < 0) {
        perror("Socket creation failed");
        exit(EXIT_FAILURE);
    }

    server_addr.sin_family = AF_INET;
    server_addr.sin_port = htons(PORT);
    server_addr.sin_addr.s_addr = inet_addr("127.0.0.1");

    // Connect to server
    if (connect(client_fd, (struct sockaddr *)&server_addr,
sizeof(server_addr)) < 0) {
        perror("Connection failed");
        exit(EXIT_FAILURE);
    }
}
```

```
printf("Connected to server.\n");

while (1) {
    printf("Enter message: ");
    fgets(buffer, BUFFER_SIZE, stdin);
    buffer[strcspn(buffer, "\n")] = '\0'; // Remove newline

    if (strcmp(buffer, "exit") == 0) {
        send(client_fd, buffer, strlen(buffer), 0);
        break;
    }

    send(client_fd, buffer, strlen(buffer), 0);

    memset(buffer, 0, BUFFER_SIZE);
    int bytes_received = recv(client_fd, buffer, BUFFER_SIZE, 0);
    if (bytes_received > 0) {
        printf("Server: %s\n", buffer);
    }
}

close(client_fd);
return 0;
}
```